

Senior Android Knowledge Base — ЧАСТЬ 2 (2026): гочи, ловушки и калибровка уровня

Главный тезис этой части: на senior-собеседовании 2026 года кандидата отделяют от middle не знанием API, а пониманием того, где **книжный ответ незаметно неверен** — от сломанного `equals()` у `data class` с массивом и `synchronized` через suspension point до того, что стектрейс ANR систематически врёт, а Android 16 молча игнорирует `screenOrientation="portrait"` на планшетах. Ниже — 10 тем, для каждой теория (не-очевидное), калибровочные вопросы с разбором Junior/Middle/Senior и 6 практических задач-ловушек.

TL;DR

- **Самые разоблачающие гочи 2026**, по темам: (1) `equals()` у `data class` с полем-массивом сравнивает массив по ссылке, а не по содержимому; (2) `synchronized` через suspension point — это баг корректности, а не перформанса, потому что монитор JVM привязан к потоку, а корутина меняет поток; (3) стектрейс ANR указывает на Activity, хотя main занял BroadcastReceiver до неё; (4) `shouldShowRequestPermissionRationale` не различает «не спрашивали» и «отказано навсегда».
- **Актуальное на платформе 2026**: Android 16 (API 36) на экранах $sw \geq 600dp$ игнорирует ограничения ориентации/resizability для приложений с targetSdk 36 (opt-out убирается в API 37); Google Play требует targeting API 36 с 31 августа 2026; permission auto-reset отзывает разрешения после ~90 дней неиспользования; PendingIntent-флаги мутабельности обязательны с API 31; KSP2 — дефолт с начала 2025, KSP1 несовместим с Kotlin 2.3.0+/AGP 9.0+.
- **Что раскрывает реальный опыт**: знание, что предупреждение о массиве в data class даёт IDE-инспекция, а не компилятор; что `Mutex` неореентрантный; что `receiveAsFlow` без закрытия канала течёт; что buildSrc инвалидирует configuration cache для всей сборки; что App Startup сам по себе не ускоряет старт, если оставить всё eager.

Key Findings

- **Kotlin:** extension-функции статически диспетчеризуются (по объявленному типу); `(data class)` с `(Array)` даёт референсное `(equals())` → нужен ручной override через `(contentEquals())`/`(contentHashCode())`; `(Mutex)` `suspension-safe` и **нереентрантный**; Channel-capacity (RENDEZVOUS/BUFFERED/CONFLATED/UNLIMITED) и различие `(consumeAsFlow)` (1 коллектор, авто-закрытие) vs `(receiveAsFlow)` (N коллекторов, ручное закрытие).
 - **Permissions:** третье состояние медиа-доступа на Android 14 (`(READ_MEDIA_VISUAL_USER_SELECTED)`); Photo Picker без разрешений; неоднозначность `(shouldShowRequestPermissionRationale)`; auto-reset ~90 дней.
 - **Утечки:** retention «объект собирается, но слишком поздно» как отдельный класс; механика LeakCanary (ObjectWatcher + KeyedWeakReference + 5c/GC + Shark); `(DisposeOnViewTreeLifecycleDestroyed)` для ComposeView в Fragment.
 - **ANR:** пороги по компонентам; финальный стектрейс вводит в заблуждение; слепые зоны StrictMode.
 - **Безопасность интенгов:** hijacking custom scheme, App Links + assetlinks.json, обязательные флаги мутабельности PendingIntent, intent redirection.
 - **ViewModel:** иерархия StoreOwner, лимит Bundle 1 МБ, что переживает process death.
 - **Compose a11y + Android 16, AndroidView interop, Gradle build internals, App Startup** — детали ниже.
-

Тема 1. Kotlin: система типов и конкурентность — ловушки

Теория

Три места, где ломается «книжное» понимание. **(1) Variance.** Declaration-site (`(out)`/`(in)` на объявлении класса) фиксирует роль параметра: `(out T)` — только producer-позиции, `(in T)` — только consumer. Use-site (проекции `(Array<out T>)`, `(Array<in T>)`) ограничивает конкретное использование там, где класс инвариантен; `(Array<out T>)` соответствует Java `(Array<? extends T>)`. Star-projection `(*>)` = «читать безопасно как upper bound, писать нельзя (кроме null)». **(2) reified** работает только в `(inline)`-функциях: тип встраивается в каждый call-site, снимая type erasure (`(x is T)`, `(T::class)` становятся доступны); `(noinline)` исключает лямбду из инлайнинга (её можно хранить/передавать), `(crossinline)` запрещает non-local return, но оставляет инлайнинг — нужно, когда лямбда вызывается из другого контекста (Runnable, другой поток). **(3) extension-функции диспетчеризуются статически** — по объявленному типу, не по runtime-типу: это не полиморфизм, а сахар над статическим методом с receiver-ом как первым аргументом.

Q1.1 (GOTCHA) — `(equals())` у data class с полем-массивом

Junior. «data class сам генерирует `(equals())`/`(hashCode())` по всем полям конструктора, так что два объекта с одинаковыми данными равны из коробки.»

Middle. «С массивом ломается: сгенерированный `equals()` для поля `Array` вызывает унаследованный от `Any` `equals()`, а это ссылочное равенство. Два инстанса с идентичным содержимым массива будут НЕ равны. Нужно вручную переопределить `equals()`/`hashCode()` через `contentEquals()`/`contentHashCode()`.»

Senior. «Корень в том, что `Array` не переопределяет `equals()` — в Kotlin `==` для массивов это референсное равенство (в отличие от `List`); официально: по умолчанию `equals()` наследуется от `Any` и реализует *referential equality*, а для сравнения элементов массивов нужен `contentEquals()`. `data class` генерирует `equals()`, только если он не написан явно, поэтому фикс — ручной `override` с `contentEquals()`/`contentHashCode()`, а для вложенных массивов `contentDeepEquals`/`contentDeepHashCode`. Критично: предупреждение об этом даёт **IDE-инспекция IntelliJ/Android Studio ‘Array property in data class’** (‘recommended to override equals() and hashCode()’), а НЕ компилятор — чистый `kotlinc`-билд соберётся молча, и на CLI-CI эту проблему легко не заметить. Практическое следствие: такой `data class` ломает дедупликацию в `Set`, ключи в `Map`, `distinctUntilChanged()` в Flow и `assertEquals` в тестах. По этой причине в domain-моделях предпочитают `List`/`immutable`-коллекции вместо массивов.»

Зачем спрашивают. Проверяют, понимает ли кандидат разницу между структурным и референсным равенством и знает ли, что `data class` не панацея. Тот, кто ловил этот баг в проде (сломанный `distinctUntilChanged`, «мигающий» UI), ответит мгновенно; книжный кандидат уверенно скажет «работает из коробки».

Q1.2 (GOTCHA) — `synchronized` через suspension point

Junior. «Для защиты общего состояния оборачиваю в `synchronized`/`@Synchronized` — как в Java, потокобезопасно.»

Middle. «В корутинах вместо `synchronized` нужен `Mutex.withLock`, потому что `Mutex` suspend-friendly: он не блокирует поток, а приостанавливает корутину.»

Senior. «Держать `synchronized`/`ReentrantLock` через suspension point — это баг корректности, а не только производительности. Монитор JVM привязан к потоку: захватывает и освобождает его один и тот же поток. Но при `suspend`-вызове (`delay`, сетевой вызов) корутина может возобновиться на ДРУГОМ потоке dispatcher-a — освобождать монитор будет не тот поток, что захватил, что нарушает семантику взаимного исключения. `Mutex` привязан к корутине (логической единице исполнения), а не к потоку, поэтому suspension-safe. Критично: `Mutex` **НЕ реентрантный** — повторный `lock()` из той же корутины, что уже держит лок, приведёт к вечному подвисанию (в отличие от реентрантного `synchronized`). Trade-off: `Mutex` заметно медленнее JVM-локов на CPU-bound операциях без suspend внутри — там лучше `atomic`-типы или обычный `lock`. Идеал по официальному гайду — вообще избегать shared mutable state: immutability, single-thread confinement через dispatcher, или actor/`StateFlow`.» `Kotlin`